



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment – 11

Map Coloring using CSP

Aim

To implement the Map Coloring problem using a **Constraint Satisfaction Problem (CSP)** in Python.

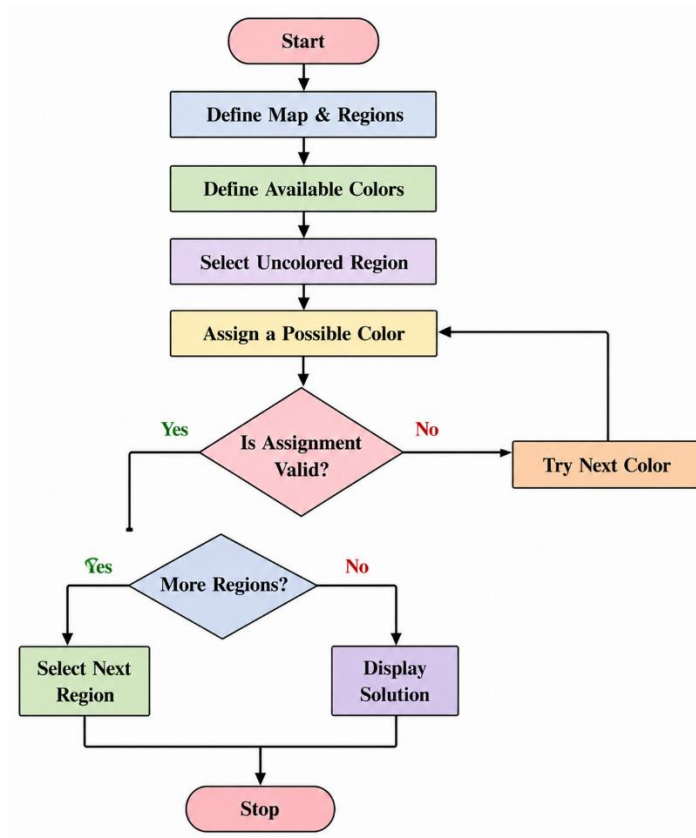
Objective

To assign different colors to the regions of a map such that no two adjacent regions have the same color.

Algorithm

1. Define the regions and their neighboring regions.
2. Define the available colors.
3. Select an uncolored region.
4. Assign a color to the selected region.
5. Check whether the color assignment satisfies all constraints.
6. If the assignment is valid, move to the next region.
7. If no valid color is available, backtrack to the previous region.
8. Continue until all regions are colored.
9. Display the solution.

FlowChart



Code

```
colors = ['Red', 'Green', 'Blue']
```

```
regions = ['A', 'B', 'C', 'D']
```

```
neighbors = {  
    'A': ['B', 'C'],  
    'B': ['A', 'C', 'D'],  
    'C': ['A', 'B', 'D'],  
    'D': ['B', 'C']  
}
```

```
assignment = {}
```

```
def is_valid(region, color):  
    for neighbor in neighbors[region]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False  
    return True
```

```
def solve():  
    if len(assignment) == len(regions):  
        return True
```

```
    # Select an uncolored region  
    for region in regions:  
        if region not in assignment:  
            break
```

```
    # Try each available color  
    for color in colors:  
        if is_valid(region, color):  
            assignment[region] = color
```

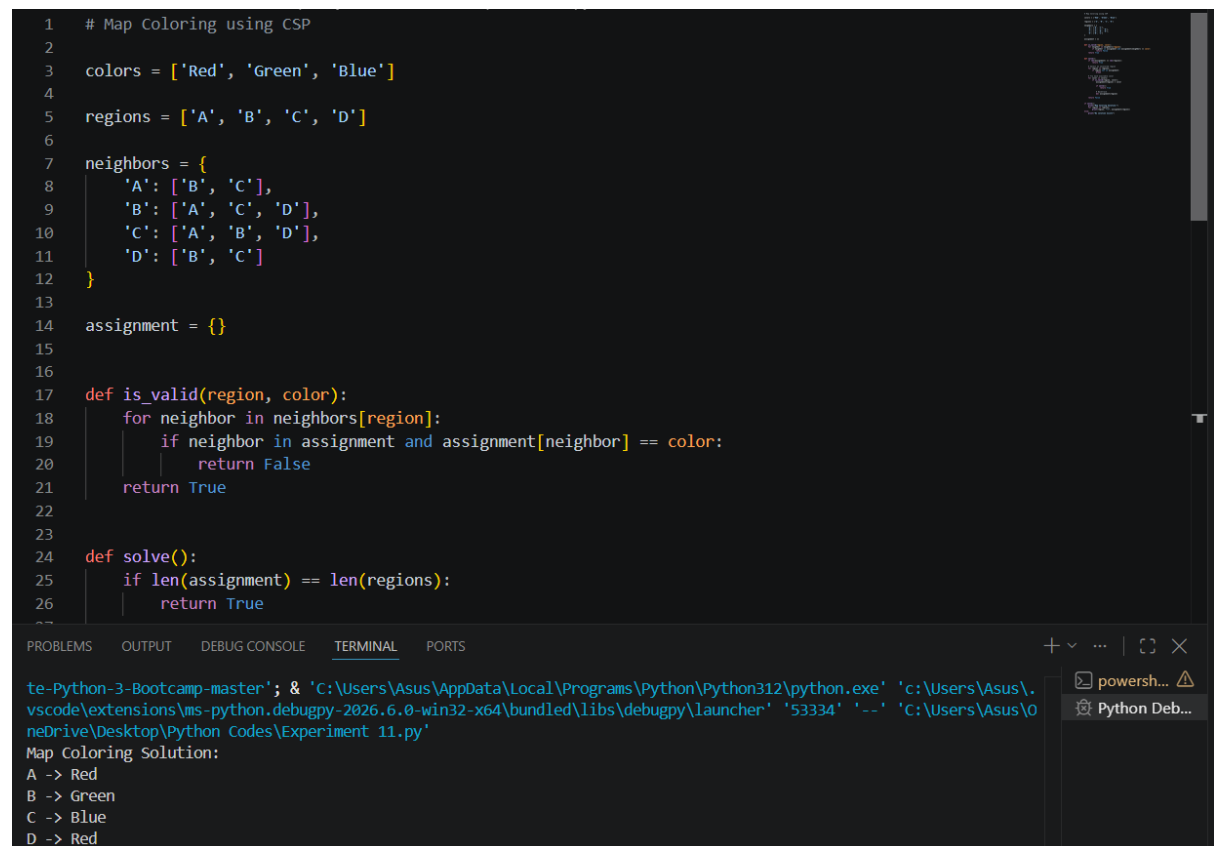
```
        if solve():  
            return True
```

```
    # Backtrack  
    del assignment[region]
```

```
    return False
```

```
if solve():  
    print("Map Coloring Solution:")  
    for region in regions:  
        print(region, "->", assignment[region])  
else:  
    print("No solution exists")
```

Output



The image shows a VS Code editor window with a Python script for Map Coloring using Constraint Satisfaction Problem (CSP) solving. The script defines colors, regions, and neighbors, then implements a solve function. The terminal output shows the solution for the map coloring problem.

```
1 # Map Coloring using CSP  
2  
3 colors = ['Red', 'Green', 'Blue']  
4  
5 regions = ['A', 'B', 'C', 'D']  
6  
7 neighbors = {  
8     'A': ['B', 'C'],  
9     'B': ['A', 'C', 'D'],  
10    'C': ['A', 'B', 'D'],  
11    'D': ['B', 'C']  
12 }  
13  
14 assignment = {}  
15  
16  
17 def is_valid(region, color):  
18     for neighbor in neighbors[region]:  
19         if neighbor in assignment and assignment[neighbor] == color:  
20             return False  
21     return True  
22  
23  
24 def solve():  
25     if len(assignment) == len(regions):  
26         return True
```

Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red

Result

The Map Coloring problem was successfully implemented using the Constraint Satisfaction Problem (CSP) approach.

Conclusion

The CSP-based program successfully assigns colors to all regions while satisfying the constraint that adjacent regions must have different colors. Backtracking is used whenever a color assignment violates a constraint.